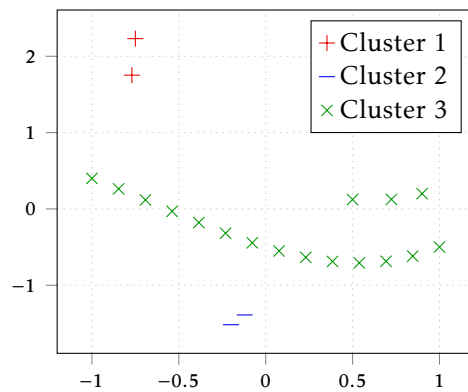


1. (A) Below is a tabulated list of sets corresponding to the resulting clusters of running a fully-fledge vanilla implementation of Agglomerative Clustering on C1.txt dataset.

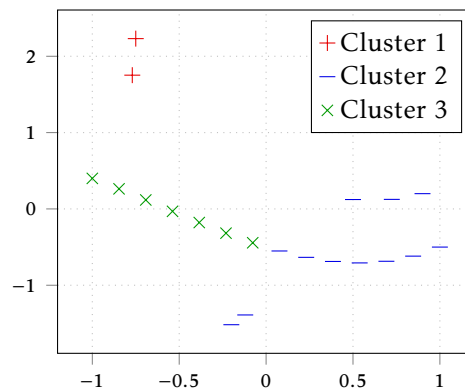
	Cluster 1	Cluster 2	Cluster 3
Single-Link	{1, 2}	{3, 4}	{5 ... 21}
Complete-Link	{1, 2}	{3, 4, 5, 6, 7, 15, 16, 17, 18, 19, 20, 21}	{8 ... 14}

Table 1. Hierarchical Agglomerative Clustering. See [Listing 4](#) for a fuller explanation on how these clusters have been obtained.

Pictorially, [Table 1](#) translates to the following plots:



(a) Single Linkage Hierarchical Clustering.



(b) Complete Linkage Hierarchical Clustering.

- (B) Below is an implementation of AgglomerativeClustering in scikit-learn:

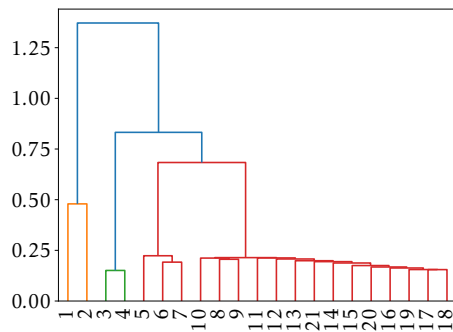
```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  from sklearn.cluster import AgglomerativeClustering
5
6  DATASET = 'data/C1.txt'
7
8  def load_dataset(dataset: str) -> np.ndarray:
9      data = np.loadtxt(dataset, usecols=(1,2))
10     return data
11
12  def hierarchical(points: np.ndarray, linkage: str, k: int = 3) -> np.ndarray:
13      clusters = AgglomerativeClustering(n_clusters=k, linkage=linkage).fit(points)
14      return clusters.labels_
15
16  def main() -> int:
17      dataset = load_dataset(DATASET)
18      s_link_labels = hierarchical(dataset, 'single')
19      c_link_labels = hierarchical(dataset, 'complete')
20      fig, (ax1, ax2) = plt.subplots(2)
21      ax1.scatter(dataset[:,0], dataset[:,1], c=s_link_labels, s=10)
22      ax2.scatter(dataset[:,0], dataset[:,1], c=c_link_labels, s=10)
23      plt.show()
24      return 0
25
26  if __name__ == '__main__':
27      raise SystemExit(main())

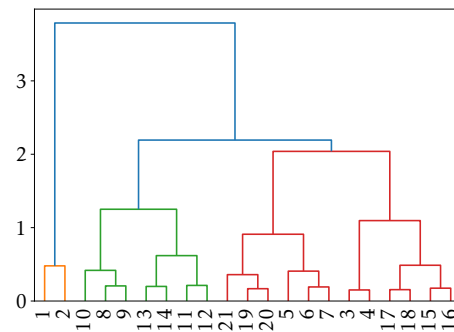
```

Listing 1. hierarchical-scikit.py

(C) The corresponding dendrograms are plotted below.



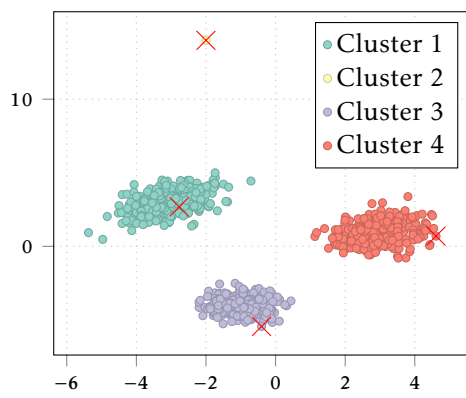
(a) Single Linkage Hierarchical Clustering.



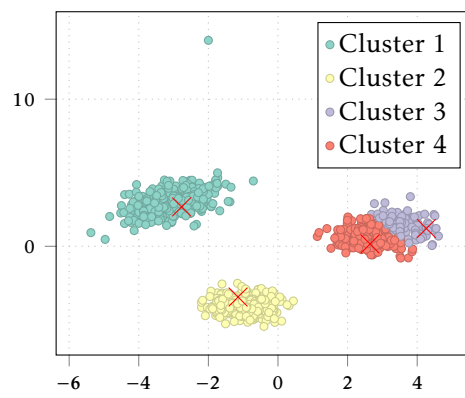
(b) Complete Linkage Hierarchical Clustering.

Figure 2. Hierarchical Clustering Dendrograms. See [Listing 5](#) for a fuller explanation on how these dendrograms have been generated.

2. (A) (i) Below is a plot corresponding to the resulting output of a fully-fledge vanilla implementation of Gonzalez and Kmeans++ clustering algorithms on C2.txt dataset.



(a) Gonzalez Clustering Algorithm.



(b) Kmeans++ Clustering Algorithm.

Figure 3. Gonzalez and Kmeans++ clustering algorithms on C2.txt dataset. Please refer to [Listing 6](#) and [7](#) for a fuller explanation on how these figures have been generated.

(ii) The following 4-center costs were obtained during the run:

	Gonzalez	Kmeans++
4-Center Cost	3.481123664686921	11.348260306157117

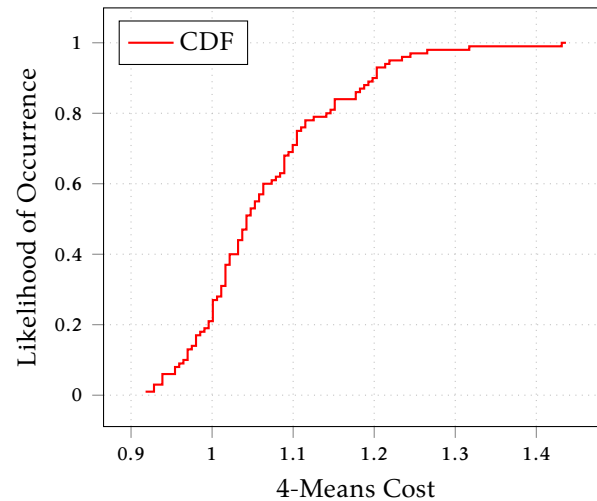
Table 2. 4-center cost.

(iii) The following 4-means costs were obtained during the run:

	Gonzalez	Kmeans++
4-Means Cost	1.6005365908052087	1.0416802543222048

Table 3. 4-means cost.

- (B) (i) The cumulative density function of the 4-means cost associated with the KMeans++ algorithm is plotted below.

**Figure 4.** 4-Means Cost CDF Plot of KMeans++ Algorithm.

- (ii) A total of $0/100 = 0.0$ of the trials, the subsets from Kmeans++ and Gonzalez matched within an error tolerance $\varepsilon < 0.01$. Please refer to [Listing 8](#) for a fuller explanation on how this fractional value was obtained.

- (C) (i) Below is the output of Lloyd's algorithm with centroids initially set as the points indexed $\{1, 2, 3, 4\}$ in the C2.txt dataset. A 4-means cost of approximately **0.8532158400114525** was obtained during the run.

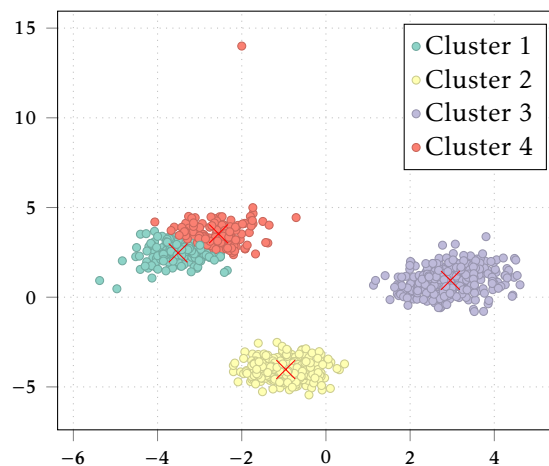


Figure 5. Lloyd's algorithm with centroids initially set as the points indexed at $\{1, 2, 3, 4\}$ on C2.txt dataset. Please refer to [Listing 9](#) for a fuller explanation on how this plot was generated.

- (ii) Below is the output of Lloyd's algorithm with centroids initially set as the output Gonzalez's subroutine. The run incurred a 4-means cost of approximately **0.8769132227723297**.

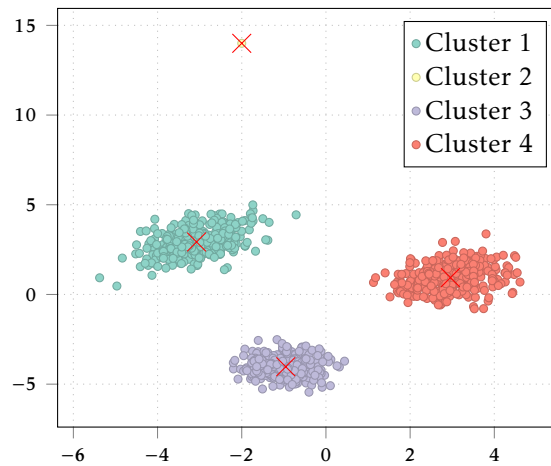


Figure 6. Lloyd's algorithm with centroids initially set as the output of Gonzalez's subroutine. Please refer to [Listing 9](#) for a fuller explanation on how this plot has been generated.

- (iii) The cumulative density function of the 4-means cost associated with a vanilla implementation of Lloyd's algorithm (found in [Listing 10](#)) is plotted below.

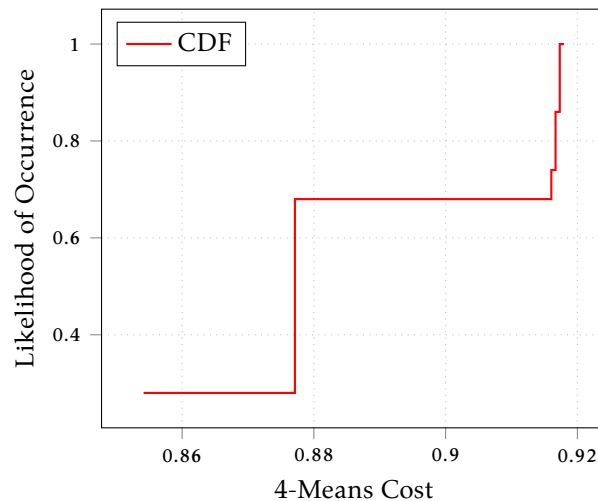


Figure 7. 4-Means Cost CDF Plot of Lloyd's Algorithm. (Vanilla Implementation).

A total of $0/100 = 0.0$ of the trials, the subsets from Kmeans++ and Lloyd matched within an error tolerance $\varepsilon < 0.01$. Please refer to [Listing 10](#) for a fuller explanation on how this fractional value was obtained.

- (D) The cumulative density function of the 4-means cost associated with a scikit-learn implementation of Lloyd's algorithm (found in [Listing 2](#)) is plotted below.

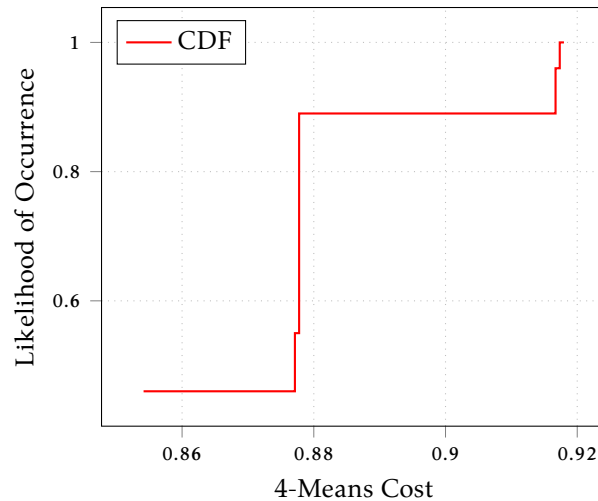


Figure 8. 4-Means Cost CDF Plot of Lloyd's Algorithm.
(Scikit-Learn Implementation).

A total of $0/100 = 0.0$ of the trials, the subsets from Kmeans++ and Lloyd matched within an error tolerance $\varepsilon < 0.01$. Please refer to [Listing 2](#) for a fuller explanation on how this fractional value was obtained.

```

1  import numpy as np
2
3  from lloyd import load_dataset, kmeans_cost
4  from sklearn.cluster import KMeans
5
6  TRIALS = 100
7  EPSILON = 0.01
8  KPLUSPLUS = 1.1120373276842588
9
10 def main() -> int:
11     dataset = load_dataset('data/C2.txt')
12     kmcosts = np.zeros(TRIALS)
13     counter = 0
14     kmeans = KMeans(n_clusters=4, init='k-means++', n_init=1)
15
16     for trial in range(TRIALS):
17         centers = kmeans.fit(dataset).cluster_centers_
18         kmcosts[trial] = kmeans_cost(dataset, centers)
19         if np.abs(kmcosts[trial] - KPLUSPLUS) < EPSILON:
20             counter += 1
21
22     np.savetxt('data/lloyd/lloyd-scikit-cdf.csv', kmcosts, delimiter=',')
23     print(counter / TRIALS)
24     return 0
25
26 if __name__ == '__main__':
27     raise SystemExit(main())

```

Listing 2. lloyd-scikit-cdf.py

3. Please refer to [Listing 11](#) for a fuller explanation on how the following centers have been generated. A cost of **0.4194209920788438** was obtained during the run. The associated .txt file has been uploaded onto google drive under the alias [u1419667-best-centers.txt](#).

1	1	1.004702	0.015617	0.008167	0.002719	-0.003609
2	2	-0.001783	0.459160	0.010494	0.518272	0.002853
3	3	0.982001	1.993834	0.001029	-0.005850	0.992876
4	4	-0.003549	-0.004410	1.016327	-0.003080	-0.009393

Listing 3. u1419667-best-centers.txt

Appendix

```

1  import numpy as np
2
3  from typing import Callable as func
4  from itertools import product, starmap
5
6  DATASET = 'data/C1.txt'
7
8  def load_dataset(dataset: str) → np.ndarray:
9      data = np.loadtxt(dataset, usecols=(1,2))
10     return data
11
12  def dist_mat(C1: np.ndarray, C2: np.ndarray) → np.ndarray:
13     return np.array(list(starmap(lambda p, q: np.linalg.norm(p - q), product(C1, C2))))
14
15  def s_link(C1: np.ndarray, C2: np.ndarray) → float:
16     return np.min(dist_mat(C1, C2))
17
18  def c_link(C1: np.ndarray, C2: np.ndarray) → float:
19     return np.max(dist_mat(C1, C2))
20
21  def hierarchical(points: np.ndarray, linkage: func, k: int = 3) → list:
22     clusters = [np.array([point]) for point in points]
23     min_dist = np.inf
24     while (num_clusters := len(clusters)) > k:
25         for i in range(num_clusters):
26             for j in range(i + 1, num_clusters):
27                 dist = linkage(clusters[i], clusters[j])
28                 if dist ≤ min_dist:
29                     min_dist = dist
30                     C1_idx = i
31                     C2_idx = j
32             clusters[C1_idx] = np.vstack((clusters[C1_idx], clusters[C2_idx]))
33             clusters.pop(C2_idx)
34             min_dist = np.inf
35     return clusters
36
37  def main() → int:
38     dataset = load_dataset(DATASET)
39     s_link_clusters = hierarchical(dataset, s_link)
40     c_link_clusters = hierarchical(dataset, c_link)
41
42     for index, cluster in enumerate(s_link_clusters, 1):
43         np.savetxt(f'data/hierarchical/s-link-cluster-{index}.csv', cluster, delimiter=',')
44     for index, cluster in enumerate(c_link_clusters, 1):
45         np.savetxt(f'data/hierarchical/c-link-cluster-{index}.csv', cluster, delimiter=',')
46
47     return 0
48
49  if __name__ == '__main__':
50     raise SystemExit(main())

```

Listing 4. hierarchical.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from scipy.cluster.hierarchy import dendrogram, linkage
5
6 plt.rcParams['font.size'] = 20
7 plt.rcParams['text.usetex'] = True
8 plt.rcParams['text.latex.preamble'] = r'\usepackage{kpfonts}'
9
10 DATASET = 'data/C1.txt'
11
12 def load_dataset(dataset: str) -> np.ndarray:
13     data = np.loadtxt(dataset, usecols=(1,2))
14     return data
15
16 def save_dendrogram(points: np.ndarray, method: str) -> None:
17     plt.cla()
18     Z = linkage(points, method=method)
19     _ = dendrogram(
20         Z,
21         leaf_label_func=lambda i: r'${s}${ % (i+1),
22         leaf_font_size=plt.rcParams.get('font.size'),
23         leaf_rotation=90,
24         count_sort=True,
25         color_threshold=(0.7 if method == 'single' else 2.1),
26     )
27     plt.tight_layout()
28     plt.savefig(f'figs/dendrogram-{method}.pdf')
29
30 def main() -> int:
31     dataset = load_dataset(DATASET)
32     save_dendrogram(dataset, 'single')
33     save_dendrogram(dataset, 'complete')
34     return 0
35
36 if __name__ == '__main__':
37     raise SystemExit(main())
```

Listing 5. hierarchical-dendrogram.py

```

1  import numpy as np
2
3  DATASET = 'data/C2.txt'
4
5  def load_dataset(dataset: str) → np.ndarray:
6      data = np.loadtxt(dataset, usecols=(1,2))
7      return data
8
9  def center_dists(points: np.ndarray, centers: np.ndarray) → np.ndarray:
10     dists = np.ones(points.shape[0]) * np.inf
11     for point_id, point in enumerate(points):
12         for center_id, center in enumerate(centers):
13             dists[point_id] = min(dists[point_id], np.linalg.norm(point - center))
14     return dists
15
16  def kcenter_cost(points: np.ndarray, centers: np.ndarray) → float:
17     dists = center_dists(points, centers)
18     return np.max(dists)
19
20  def kmeans_cost(points: np.ndarray, centers: np.ndarray) → float:
21     dists = center_dists(points, centers)
22     return np.sqrt(np.mean(np.square(dists)))
23
24  def max_dist_point(points: np.ndarray, centers: np.ndarray) → np.ndarray:
25     dists = center_dists(points, centers)
26     return points[dists.argmax()]
27
28  def gonzalez(points: np.ndarray, k: int = 4) → np.ndarray:
29     centers = np.array([points[0]])
30     while len(centers) < k:
31         centers = np.vstack((centers, max_dist_point(points, centers)))
32     return centers
33
34  def query_labels(points: np.ndarray, centers: np.ndarray) → np.ndarray:
35     labels, dists = np.zeros(points.shape[0]), np.ones(points.shape[0]) * np.inf
36     for point_id, point in enumerate(points):
37         for center_id, center in enumerate(centers):
38             if (curr_dist := np.linalg.norm(point - center)) < dists[point_id]:
39                 dists[point_id], labels[point_id] = curr_dist, center_id
40     return labels
41
42  def main() → int:
43     dataset = load_dataset(DATASET)
44     centers = gonzalez(dataset)
45     clabels = query_labels(dataset, centers)
46
47     np.savetxt('data/gonzalez/centers.csv', centers, delimiter=',')
48     np.savetxt('data/gonzalez/points.csv', np.column_stack((dataset, clabels)), delimiter=',')
49
50     print(kcenter_cost(dataset, centers))
51     print(kmeans_cost(dataset, centers))
52
53     return 0
54
55  if __name__ == '__main__':
56     raise SystemExit(main())

```

Listing 6. gonzalez.py

```

1  import numpy as np
2
3  DATASET = 'data/C2.txt'
4
5  np.random.seed(0)
6
7  def load_dataset(dataset: str) → np.ndarray:
8      data = np.loadtxt(dataset, usecols=(1,2))
9      return data
10
11 def center_dists(points: np.ndarray, centers: np.ndarray) → np.ndarray:
12     dists = np.ones(points.shape[0]) * np.inf
13     for point_id, point in enumerate(points):
14         for center_id, center in enumerate(centers):
15             dists[point_id] = min(dists[point_id], np.linalg.norm(point - center))
16     return dists
17
18 def kcenter_cost(points: np.ndarray, centers: np.ndarray) → float:
19     dists = center_dists(points, centers)
20     return np.max(dists)
21
22 def kmeans_cost(points: np.ndarray, centers: np.ndarray) → float:
23     dists = center_dists(points, centers)
24     return np.sqrt(np.mean(np.square(dists)))
25
26 def kplusplus(points: np.ndarray, k: int = 4) → np.ndarray:
27     centers = np.array([points[0]])
28     for _ in range(k-1):
29         dists = center_dists(points, centers)
30         probs = np.square(dists) / np.sum(np.square(dists))
31         index = np.random.choice(points.shape[0], p=probs)
32         centers = np.vstack((centers, points[index]))
33     return centers
34
35 def query_labels(points: np.ndarray, centers: np.ndarray) → np.ndarray:
36     labels, dists = np.zeros(points.shape[0]), np.ones(points.shape[0]) * np.inf
37     for point_id, point in enumerate(points):
38         for center_id, center in enumerate(centers):
39             if (curr_dist := np.linalg.norm(point - center)) < dists[point_id]:
40                 dists[point_id], labels[point_id] = curr_dist, center_id
41     return labels
42
43 def main() → int:
44     dataset = load_dataset(DATASET)
45     centers = kplusplus(dataset)
46     clabels = query_labels(dataset, centers)
47
48     np.savetxt('data/kplusplus/centers.csv', centers, delimiter=',')
49     np.savetxt('data/kplusplus/points.csv', np.column_stack((dataset, clabels)), delimiter=',')
50
51     print(kcenter_cost(dataset, centers))
52     print(kmeans_cost(dataset, centers))
53
54     return 0
55
56 if __name__ == '__main__':
57     raise SystemExit(main())

```

Listing 7. kplusplus.py

```
1  import numpy as np
2
3  from kplusplus import load_dataset, kplusplus, kmeans_cost
4
5  TRIALS = 100
6  EPSILON = 0.01
7  GONZALEZ = 1.6005365908052087
8
9  np.random.seed(0)
10
11 def main() -> int:
12     dataset = load_dataset('data/C2.txt')
13     kmcosts = np.zeros(TRIALS)
14     counter = 0
15
16     for trial in range(TRIALS):
17         centers = kplusplus(dataset)
18         kmcosts[trial] = kmeans_cost(dataset, centers)
19         if np.abs(kmcosts[trial] - GONZALEZ) < EPSILON:
20             counter += 1
21
22     np.savetxt('data/kplusplus/kplusplus-cdf.csv', kmcosts, delimiter=',')
23     print(counter / TRIALS)
24     return 0
25
26 if __name__ == '__main__':
27     raise SystemExit(main())
```

Listing 8. kplusplus-cdf.py

```

1  import numpy as np
2
3  from gonzalez import gonzalez
4
5  DATASET = 'data/C2.txt'
6
7  def load_dataset(dataset: str) → np.ndarray:
8      data = np.loadtxt(dataset, usecols=(1,2))
9      return data
10
11 def center_dists(points: np.ndarray, centers: np.ndarray) → np.ndarray:
12     dists = np.ones(points.shape[0]) * np.inf
13     for point_id, point in enumerate(points):
14         for center_id, center in enumerate(centers):
15             dists[point_id] = min(dists[point_id], np.linalg.norm(point - center))
16     return dists
17
18 def kmeans_cost(points: np.ndarray, centers: np.ndarray) → float:
19     dists = center_dists(points, centers)
20     return np.sqrt(np.mean(np.square(dists)))
21
22 def lloyd(points: np.ndarray, k: int = 4, init_centers: np.ndarray | None = None, num_iter: int = 100) → np.
    ndarray:
23     centers = init_centers if init_centers is not None else points[:k]
24     for _ in range(num_iter):
25         dists = np.linalg.norm(points[:, np.newaxis] - centers, axis=2)
26         index = np.argmin(dists, axis=1)
27         for center_id in range(k):
28             centers[center_id] = np.mean(points[index == center_id], axis=0)
29     return centers
30
31 def query_labels(points: np.ndarray, centers: np.ndarray) → np.ndarray:
32     labels, dists = np.zeros(points.shape[0]), np.ones(points.shape[0]) * np.inf
33     for point_id, point in enumerate(points):
34         for center_id, center in enumerate(centers):
35             if (curr_dist := np.linalg.norm(point - center)) < dists[point_id]:
36                 dists[point_id], labels[point_id] = curr_dist, center_id
37     return labels
38
39 def main() → int:
40     dataset = load_dataset(DATASET)
41     all_centers = [
42         ('lloyd', lloyd(dataset)),
43         ('gonzalez', lloyd(dataset, init_centers=gonzalez(dataset)))
44     ]
45
46     for algorithm, centers in all_centers:
47         clabels = query_labels(dataset, centers)
48         np.savetxt(f'data/lloyd/centers-{algorithm}.csv', centers, delimiter=',')
49         np.savetxt(f'data/lloyd/points-{algorithm}.csv', np.column_stack((dataset, clabels)), delimiter=',')
50         print(algorithm, kmeans_cost(dataset, centers))
51
52     return 0
53
54 if __name__ == '__main__':
55     raise SystemExit(main())

```

Listing 9. lloyd.py

```
1 import numpy as np
2
3 from kplusplus import kplusplus
4 from lloyd import load_dataset, kmeans_cost, lloyd
5
6 TRIALS = 100
7 EPSILON = 0.01
8 KPLUSPLUS = 1.1120373276842588
9
10 def main() -> int:
11     dataset = load_dataset('data/C2.txt')
12     kmcosts = np.zeros(TRIALS)
13     counter = 0
14
15     for trial in range(TRIALS):
16         centers = lloyd(dataset, init_centers=kplusplus(dataset))
17         kmcosts[trial] = kmeans_cost(dataset, centers)
18         if np.abs(kmcosts[trial] - KPLUSPLUS) < EPSILON:
19             counter += 1
20
21     np.savetxt('data/lloyd/lloyd-cdf.csv', kmcosts, delimiter=',')
22     print(counter / TRIALS)
23     return 0
24
25 if __name__ == '__main__':
26     raise SystemExit(main())
```

Listing 10. lloyd-cdf.py

```

1  import numpy as np
2
3  from kplusplus import kplusplus
4
5  DATASET = 'data/C3.txt'
6
7  np.random.seed(0)
8
9  def load_dataset(dataset: str) → np.ndarray:
10     data = np.loadtxt(dataset, usecols=(1,2))
11     return data
12
13  def center_dists(points: np.ndarray, centers: np.ndarray) → np.ndarray:
14     dists = np.ones(points.shape[0]) * np.inf
15     for point_id, point in enumerate(points):
16         for center_id, center in enumerate(centers):
17             dists[point_id] = min(dists[point_id], np.linalg.norm(point - center))
18     return dists
19
20  def kmedians_cost(points: np.ndarray, centers: np.ndarray) → float:
21     dists = center_dists(points, centers)
22     return np.mean(dists)
23
24  def kmedians(points: np.ndarray, k: int = 4, init_centers: np.ndarray | None = None, num_iter: int = 100) →
    np.ndarray:
25     centers = init_centers if init_centers is not None else points[:k]
26     for _ in range(num_iter):
27         dists = np.linalg.norm(points[:, np.newaxis] - centers, axis=2)
28         index = np.argmin(dists, axis=1)
29         for center_id in range(k):
30             centers[center_id] = np.median(points[index == center_id], axis=0)
31     return centers
32
33  def main() → int:
34     dataset = load_dataset(DATASET)
35     centers = kmedians(dataset, init_centers=kplusplus(dataset))
36     kmecost = kmedians_cost(dataset, centers)
37     indices = np.arange(centers.shape[0]) + 1
38
39     np.savetxt('data/kmedians/best-centers.txt', np.column_stack((indices, centers)), delimiter='\t', fmt=['%d',
40         '%5f', '%5f', '%5f', '%5f', '%5f'])
41
42     print(kmecost)
43     return 0
44
45  if __name__ == '__main__':
46     raise SystemExit(main())

```

Listing 11. kmedians.py